

SO-101 机械臂 ACT 模型实验手册

数据采集 - AutoDL 云端训练 - 本地部署推理

适用于已完成 LeRobot + SO-101 基础遥操作实验的学生

使用前说明

本手册默认学生已经完成 SO-101 机械臂的 LeRobot 环境搭建、端口识别、权限授权、校准、遥操作和基础录制测试。本手册继续完成 ACT 模型实验，包括正式采集数据、上传 AutoDL、云端训练和本地部署。示例任务为 Put the banana on the plate., 示例服务器信息为 38671 / connect.westb.seetacloud.com / root，实际实验时请在对应命令处替换为自己的任务和自己的 AutoDL 信息。

目录

- 1. 实验目标与整体流程
- 2. 前置条件与实验准备
- 3. 任务设计与命令替换原则
- 4. 本地数据采集前检查
- 5. 遥操作验证与测试录制
- 6. 正式采集 SO-101 示教数据
- 7. 数据检查、压缩与上传
- 8. AutoDL 云端环境配置
- 9. ACT 模型训练
- 10. 下载模型到本地
- 11. 本地 ACT 推理部署

1. 实验目标与整体流程

本实验目标是打通真实机器人模仿学习的完整闭环：使用 SO-101 leader-follower 机械臂采集示教数据，将数据上传到 AutoDL 云服务器，在云端训练 ACT 模型，再把训练好的模型下载回本地电脑，用 SO-101 follower 机械臂进行真实推理部署。

本实验不重新讲解 LeRobot 和 SO-101 的从零搭建流程，而是在已经能够完成基础遥操作和本地录制的基础上，进一步完成 ACT 模型训练与部署。

阶段	地点	主要操作	核心命令
数据采集	本地 Ubuntu 电脑	连接 follower、leader 和摄像头，使用遥操作采集示教数据	lerobot-record
数据上传	本地 -> AutoDL	压缩数据集并通过 scp 上传到云端	tar、scp、ssh
云端训练	AutoDL 云服务器	安装训练依赖，使用数据集训练 ACT 策略	lerobot-train
模型下载	AutoDL -> 本地	找到 pretrained_model 并下载到本地	find、tar、scp
本地部署	本地 Ubuntu 电脑	只连接 follower 和摄像头，运行 ACT 模型推理	lerobot-rollout

2. 前置条件与实验准备

2.1 学生应已掌握的内容

- 能够在本地 Ubuntu 电脑执行 conda activate lerobot 并进入 LeRobot 环境。
- 能够使用 lerobot-find-port 找到 SO-101 follower 和 leader 的串口。
- 能够使用 lerobot-calibrate 完成 follower 和 leader 校准。
- 能够使用 lerobot-teleoperate 完成 leader 控制 follower 的遥操作。
- 能够使用 lerobot-find-cameras opencv 或 OpenCV 测试摄像头编号。
- 已经知道 Ctrl+C 可以停止正在运行的机器人控制程序。

2.2 本地硬件与环境

项目	要求	说明
操作系统	Ubuntu 22.04	与前置 SO-101 实验保持一致。
Python 环境	conda + lerobot 环境	建议使用前置实验中已经搭建好的 lerobot 环境。
机械臂	SO-101 follower + SO-101 leader	采集数据时需要两条机械臂；部署推理时只需要 follower。
摄像头	推荐两路 USB 摄像头	示例使用 front 和 side 两路摄像头。

磁盘空间	建议 50GB 以上	正式采集视频数据会占用较多空间。
------	------------	------------------

2.3 AutoDL 云服务器准备

- 已经拥有 AutoDL 账号，并能创建带 NVIDIA GPU 的实例。
- 建议选择带 CUDA/PyTorch/Conda 的镜像，后续环境配置会更简单。
- 需要从 AutoDL 页面复制自己的 SSH 登录信息，包括端口、用户名和服务器地址。
- 示例中的 38671、root、connect.westb.seetacloud.com 只是示例，不能直接照抄。

3. 任务设计与命令替换原则

3.1 先确定自己的任务

本文使用示例任务 Put the banana on the plate.，即“把香蕉放到盘子上”。这个任务只是示例，真实实验中每个同学可以做不同任务，例如抓取方块、把方块放进碗中、把物体移动到指定区域等。

任务示例	是否适合初次实验	说明
Put the banana on the plate.	适合	本文示例任务，动作目标明确。
Pick up the cube.	适合	单物体抓取，便于判断成功或失败。
Put the cube into the bowl.	适合	比单纯抓取稍难，但流程仍然清楚。
Sort multiple objects by color.	不建议初次使用	状态多、数据量要求高，初学者容易失败。

关键注意：任务不同时要同步替换

如果你做的不是“把香蕉放到盘子上”，需要把手册命令中的任务文本 Put the banana on the plate. 替换为你的任务文本；同时建议把 so101_banana_plate 这类数据集名、压缩包名、训练输出目录名也替换成与你任务对应的名称，例如 so101_cube_bowl。最容易漏改的位置是采集命令的 --dataset.single_task 和部署命令的 --task。

3.2 命名建议

为了避免不同任务的数据和模型混在一起，建议数据集目录和训练输出目录都体现任务名称。下面是示例替换关系：

如果任务是	任务文本建议写成	数据集目录建议写成	训练输出目录建议写成
把香蕉放到盘子上	Put the banana on the plate.	so101_banana_plate	act_so101_banana_plate
把方块放进碗中	Put the cube into the bowl.	so101_cube_bowl	act_so101_cube_bowl
抓起方块	Pick up the cube.	so101_pick_cube	act_so101_pick_cube

3.3 示教数据质量原则

- 只保留成功轨迹：失败、碰撞、抓空、物体掉落或明显偏离目标的 episode 应取消重录。
- 保持初始状态一致：每条 episode 开始时，物体、目标容器、机械臂初始姿态尽量一致。
- 保持摄像头固定：训练和部署时摄像头名称、位置、朝向、分辨率和帧率尽量一致。
- 动作稳定连续：操作 leader 时不要大幅抖动，不要突然快速甩动关节。
- 不要混合任务：一个数据集内尽量只包含一个任务。

4. 本地数据采集前检查

4.1 激活 LeRobot 环境并创建工作目录

```
conda activate lerobot

mkdir -p $HOME/lerobot_work/data
mkdir -p $HOME/lerobot_work/logs
mkdir -p $HOME/lerobot_work/models
cd $HOME/lerobot_work

which lerobot-record
which lerobot-teleoperate
which lerobot-rollout
python --version
```

如果 which lerobot-rollout 没有输出路径，说明本地环境可能缺少部署相关脚本，可以补装：

```
pip install 'lerobot[core_scripts,hardware,feetech,training]'
which lerobot-rollout
```

4.2 确认 follower 和 leader 端口

连接 follower 和 leader 的 USB，给控制板和电机上电，然后执行端口识别。

```
conda activate lerobot
lerobot-find-port
```

根据终端提示插拔设备，记录 follower 和 leader 分别对应的端口。常见端口包括 /dev/ttyACM0、/dev/ttyACM1、/dev/ttyUSB0、/dev/ttyUSB1。

端口注意点

后续命令中的 /dev/ttyACM1 和 /dev/ttyACM0 是示例。你的电脑可能刚好相反，也可能是 /dev/ttyUSB0 和 /dev/ttyUSB1。只要端口不同，就要把后续所有命令里的端口替换为自己的实际端口。

4.3 授权串口权限

下面示例假设 follower 是 /dev/ttyACM1，leader 是 /dev/ttyACM0。若你的端口不同，请替换为实际端口。

```
sudo chmod 666 /dev/ttyACM1 /dev/ttyACM0
ls -l /dev/ttyACM1 /dev/ttyACM0
```

如果后续出现 Permission denied，优先重新运行 lerobot-find-port 确认端口是否变化，然后重新执行 chmod。

4.4 确认或重新校准

如果前置实验已经用 so101_follower_01 和 so101_leader_01 完成校准，可以继续使用。若命令提示缺少 calibration，或机械臂动作方向明显异常，需要重新校准。

```
# follower 校准，端口按自己的实际结果修改
lerobot-calibrate \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM1 \
  --robot.id=so101_follower_01

# leader 校准，端口按自己的实际结果修改
lerobot-calibrate \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM0 \
  --teleop.id=so101_leader_01
```

4.5 确认摄像头编号

```
v4l2-ctl --list-devices
ls -l /dev/video*

conda activate lerobot
lerobot-find-cameras opencv
```

也可以用 OpenCV 测试 0-5 号摄像头：

```
python - <<'EOF'
import cv2
for i in range(6):
    cap = cv2.VideoCapture(i)
    ok = cap.isOpened()
    ret, frame = cap.read() if ok else (False, None)
    print(i, "opened=", ok, "read=", ret,
          "shape=", None if frame is None else frame.shape)
    cap.release()
EOF
```

本文示例使用 front 摄像头 index_or_path: 0，side 摄像头 index_or_path: 2。如果你的摄像头编号不同，后续所有 --robot.cameras 中的 index_or_path 都要修改。

5. 遥操作验证与测试录制

5.1 无摄像头遥操作测试

正式录制前，先只做遥操作测试，确认 follower 能稳定跟随 leader。下面命令中的端口按自己的实际结果修改。

```
conda activate lerobot
```

```
lerobot-teleoperate \  
--robot.type=so101_follower \  
--robot.port=/dev/ttyACM1 \  
--robot.id=so101_follower_01 \  
--teleop.type=so101_leader \  
--teleop.port=/dev/ttyACM0 \  
--teleop.id=so101_leader_01
```

- 如果 follower 不动，检查端口、权限、电源和校准。
- 如果 follower 方向明显不对，停止程序并重新校准。
- 如果机械臂有撞限位趋势，立即 Ctrl+C 停止。
- 遥操作不稳定时不要继续采集数据。

5.2 带摄像头遥操作测试

确认无摄像头遥操作正常后，再加入摄像头配置。

```
lerobot-teleoperate \  
--robot.type=so101_follower \  
--robot.port=/dev/ttyACM1 \  
--robot.id=so101_follower_01 \  
--robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, side: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" \  
--teleop.type=so101_leader \  
--teleop.port=/dev/ttyACM0 \  
--teleop.id=so101_leader_01
```

如果摄像头打不开，先重新运行 `lerobot-find-cameras opencv`，确认 `index_or_path` 是否正确。

5.3 测试录制 1-3 条数据

正式采集 50 条前，建议先录制 1-3 条测试数据，确认数据目录能正常生成。测试通过后，可以删除测试目录，再进行正式采集。

```
cd $HOME/lerobot_work  
mkdir -p data/so101_banana_plate_test  
  
lerobot-record \  
--robot.type=so101_follower \  
--robot.port=/dev/ttyACM1 \  
--robot.id=so101_follower_01 \  
--robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, side: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" \  
--teleop.type=so101_leader \  
--teleop.port=/dev/ttyACM0 \  
--teleop.id=so101_leader_01 \  
--dataset.repo_id=local/so101_banana_plate_test \  
--dataset.root=$HOME/lerobot_work/data/so101_banana_plate_test \  
--dataset.push_to_hub=False \  
--dataset.num_episodes=3 \  
--dataset.single_task="Put the banana on the plate." \  
--dataset.episode_time_s=30 \  
--dataset.reset_time_s=15 \  
--dataset.streaming_encoding=true \  
--dataset.encoder_threads=2 \  
--dataset.fps=30
```

测试录制注意点

如果你的任务不是香蕉放盘子，这里的 `so101_banana_plate_test` 和 `Put the banana on the plate`. 也要替换成你自己的任务。测试录制只是为了确认流程，不建议直接把低质量测试数据并入正式训练集。

6. 正式采集 SO-101 示教数据

6.1 正式采集命令

下面命令默认采集 50 条 episode，并保存到 `$HOME/lerobot_work/data/so101_banana_plate`。请先根据自己的端口、摄像头编号和任务修改命令。

```
conda activate lerobot
cd $HOME/lerobot_work
mkdir -p data/so101_banana_plate

lerobot-record \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM1 \
  --robot.id=so101_follower_01 \
  --robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, side: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM0 \
  --teleop.id=so101_leader_01 \
  --dataset.repo_id=local/so101_banana_plate \
  --dataset.root=$HOME/lerobot_work/data/so101_banana_plate \
  --dataset.push_to_hub=False \
  --dataset.num_episodes=50 \
  --dataset.single_task="Put the banana on the plate." \
  --dataset.episode_time_s=30 \
  --dataset.reset_time_s=15 \
  --dataset.streaming_encoding=true \
  --dataset.encoder_threads=2 \
  --dataset.fps=30
```

正式采集前只需重点改这些

1. 端口： `/dev/ttyACM1` 和 `/dev/ttyACM0` 按自己的 follower/leader 端口修改。2. 摄像头： `index_or_path: 0` 和 `2` 按自己的摄像头编号修改。3. 任务：如果不是香蕉放盘子，把 `so101_banana_plate` 和 `Put the banana on the plate`. 改成自己的任务名称和任务文本。

6.2 录制按键

按键	作用	使用场景
n 或右方向键	提前结束当前 episode 或 reset	任务已经完成，不想等到 30 秒自动结束。
r 或左方向键	取消当前 episode 并重新录制	抓取失败、碰撞、动作明显错误、物体掉

		落。
q 或 ESC	停止录制并保存已完成数据	已经完成采集，或需要中止实验。

6.3 采集过程要求

- 每条 episode 开始前，将物体和机械臂恢复到接近设定的初始状态。
- 完成任务后不要继续做无意义晃动，可以按 n 提前进入下一条。
- 失败轨迹应按 r 取消，不要为了凑数量保留失败数据。
- 采集中不要移动摄像头，不要改变桌面布置，不要切换任务。
- 建议先完成 50 条高质量数据，再考虑是否增加到 100 条。

7. 数据检查、压缩与上传

7.1 检查数据是否保存成功

```
du -sh $HOME/lerobot_work/data/so101_banana_plate
find $HOME/lerobot_work/data/so101_banana_plate -maxdepth 3 -type f | head -n 30
find $HOME/lerobot_work/data/so101_banana_plate -maxdepth 2 -type d
```

如果数据目录只有几 KB，或 find 命令几乎看不到文件，说明录制没有正常保存，需要回到第 6 章重新采集。若你的数据集目录不是 so101_banana_plate，请把命令中的目录名替换成自己的目录。

7.2 压缩数据集

数据集可能包含大量小文件和视频，推荐先压缩后上传。

```
cd $HOME/lerobot_work/data
tar -czf so101_banana_plate.tar.gz so101_banana_plate
du -sh so101_banana_plate.tar.gz
```

7.3 上传到 AutoDL

打开 AutoDL 控制台，复制自己的 SSH 登录信息。下面命令中的端口、用户名和服务器地址只是示例。

```
# 本地电脑执行：先测试能否登录云端
ssh -p 38671 root@connect.westb.seetacloud.com
```

测试能登录后，退出云端回到本地，上传压缩包。注意 scp 指定端口使用大写 -P。

```
cd $HOME/lerobot_work/data

scp -P 38671 \
  so101_banana_plate.tar.gz \
  root@connect.westb.seetacloud.com:/root/
```

AutoDL 连接信息注意点

38671、root、connect.westb.seetacloud.com 是示例。每个同学的 AutoDL 端口和地址可能不同，必须替换成自己实例页面显示的信息。ssh 登录用小写 -p，scp 上传和下载用大写 -P。

7.4 云端解压数据集

登录 AutoDL 云服务器后执行：

```
cd /root
tar -xzf so101_banana_plate.tar.gz

du -sh /root/so101_banana_plate
find /root/so101_banana_plate -maxdepth 3 -type f | head -n 30
```

如果你的任务目录不是 so101_banana_plate，需要把上面的压缩包名和目录名替换为自己的名称。

8. AutoDL 云端环境配置

8.1 检查 GPU

```
nvidia-smi
```

如果 nvidia-smi 无法运行，说明当前实例可能没有 GPU，或镜像/驱动异常，不建议继续训练。

8.2 初始化 conda 并创建 LeRobot 环境

```
conda init bash
source ~/.bashrc

conda create -n lerobot python=3.12 -y
conda activate lerobot
python --version
```

如果 conda: command not found，通常说明当前镜像没有 conda，建议更换为带 PyTorch/Conda 的镜像。

8.3 安装 FFmpeg 和 LeRobot 训练依赖

```
# AutoDL 通常是 root 用户；如果 sudo 不可用，可以去掉 sudo
sudo apt update
sudo apt install -y ffmpeg git

conda activate lerobot
python -m pip install --upgrade pip
pip install 'lerobot[training]'
```

如果 pip 下载较慢，可以先设置 pip 镜像源：

```
pip config set global.index-url https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
pip install 'lerobot[training]'
```

8.4 检查 PyTorch、CUDA 和 LeRobot

```
python - <<'EOF'
import sys
import torch
import lerobot

print("python:", sys.version)
print("torch:", torch.__version__)
print("cuda available:", torch.cuda.is_available())
print("cuda device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("gpu:", torch.cuda.get_device_name(0))
print("lerobot import: OK")
EOF

which lerobot-train
lerobot-train --help | head -n 40
```

如果 cuda available 为 False，先不要训练，应检查实例是否真的有 GPU，或更换合适的 PyTorch/CUDA 镜像。

9. ACT 模型训练

9.1 使用 screen 防止 SSH 断开

云端训练时间较长，建议使用 screen。这样即使本地 SSH 断开，只要服务器没有关机，训练进程通常仍会继续运行。

```
screen -S act_train
conda activate lerobot
```

从 screen 中临时退出：按 Ctrl+A，然后按 D。重新进入：

```
screen -ls
screen -r act_train
```

9.2 启动 ACT 训练

下面命令使用 /root/so101_banana_plate 数据集训练 ACT，并将结果保存到

/root/outputs/train/act_so101_banana_plate。若你的任务不是香蕉放盘子，请替换数据集路径、repo_id 和 output_dir。

```
conda activate lerobot

lerobot-train \
  --dataset.root=/root/so101_banana_plate \
  --dataset.repo_id=local/so101_banana_plate \
  --policy.type=act \
  --output_dir=/root/outputs/train/act_so101_banana_plate \
  --policy.device=cuda \
  --wandb.enable=false \
```

```
--policy.push_to_hub=false
```

默认训练步数、batch size 和 checkpoint 保存间隔可能随 LeRobot 版本不同而变化，实际以训练日志和输出目录为准。

9.3 观察训练状态

可以新开一个 SSH 窗口登录同一台 AutoDL 服务器，执行：

```
watch -n 1 nvidia-smi
```

正常训练时，nvidia-smi 中应能看到 python 进程占用 GPU 显存，GPU 利用率不应长期为 0。如果出现 CUDA out of memory，可考虑关闭其他进程、换更大显存 GPU，或根据 lerobot-train --help 中的参数降低 batch size。

9.4 查找训练得到的模型

```
find /root/outputs/train/act_so101_banana_plate -type d -name "pretrained_model"
```

可能会找到多个 checkpoint，例如 020000、040000、060000。第一次真实部署可以先下载较早 checkpoint 做安全测试，再根据效果尝试更高步数 checkpoint。

10. 下载模型到本地

10.1 云端压缩 pretrained_model

假设第 9 章 find 命令找到的模型路径为

/root/outputs/train/act_so101_banana_plate/checkpoints/020000/pretrained_model，则在云端执行：

```
cd /root/outputs/train/act_so101_banana_plate/checkpoints/020000
tar -czf /root/act_so101_banana_plate_020000.tar.gz pretrained_model
ls -lh /root/act_so101_banana_plate_020000.tar.gz
```

如果你选择的是其他 checkpoint，或任务名称不是 so101_banana_plate，需要把路径和压缩包名称替换为自己的实际名称。

10.2 本地下载并解压模型

回到本地电脑终端，使用 scp 下载模型。端口和服务器地址仍然必须使用自己的 AutoDL 信息。

```
mkdir -p $HOME/lerobot_work/models/so101_banana_plate_020000

scp -P 38671 \
  root@connect.westb.seetacloud.com:/root/act_so101_banana_plate_020000.tar.gz \
  $HOME/lerobot_work/models/

cd $HOME/lerobot_work/models
tar -xzf act_so101_banana_plate_020000.tar.gz -C so101_banana_plate_020000

find $HOME/lerobot_work/models/so101_banana_plate_020000 -type d -name "pretrained_model"
```

下载模型注意点

scp 下载也要使用自己的 AutoDL 端口和地址。如果你的任务不是香蕉放盘子，压缩包名、模型目录名和后续部署中的 policy.path 都要改成自己的模型名称。

11. 本地 ACT 推理部署

11.1 部署前安全检查

- 部署推理时不需要 leader，建议拔掉 leader 的 USB，避免端口混淆。
- 只连接 SO-101 follower、摄像头和电源。
- 确认 follower 周围没有手、线材和易碎物体。
- 确认物体初始位置、机械臂初始姿态和摄像头位置与采集数据时接近。
- 第一次部署时 duration 设置短一些，例如 10 或 20 秒。
- 随时准备按 Ctrl+C 停止程序；如有危险动作，必要时直接断电。

11.2 重新确认 follower 端口和摄像头

```
conda activate lerobot  
  
lerobot-find-port  
lerobot-find-cameras opencv
```

如果部署时 follower 端口或摄像头编号和采集时不同，需要同步修改 rollout 命令。部署时 camera 名称仍建议保持 front 和 side，不要改成其他名称。

11.3 运行 ACT 推理

下面命令使用下载到本地的 020000 checkpoint 进行推理。请根据自己的端口、摄像头编号、模型路径和任务文本修改。

```
conda activate lerobot  
  
lerobot-rollout \  
  --strategy.type=base \  
  --policy.path=$HOME/lerobot_work/models/so101_banana_plate_020000/pretrained_model \  
  --robot.type=so101_follower \  
  --robot.id=so101_follower_01 \  
  --robot.port=/dev/ttyACM1 \  
  --robot.cameras="{front: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30},side: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" \  
  --task="Put the banana on the plate." \  
  --duration=20
```

部署命令中最容易漏改的地方

如果你的任务不是香蕉放盘子，这里至少要改三处：1. --policy.path 中的模型目录；2. --task 中的任务文本；3. 摄像头编号和 follower 端口。尤其注意 --task 应与采集时的 --dataset.single_task 保持一致或高度一致。

11.4 推理效果判断

- 程序能成功加载模型和摄像头，说明部署链路基本打通。
- follower 能产生连续动作，说明模型输出已成功发送到真实机械臂。
- 如果第一次不能完全成功，但机械臂有接近目标、尝试抓取、尝试移动等趋势，说明模型可能已学到部分行为。
- 如果动作完全随机，优先检查摄像头位置、任务文本、模型路径和数据质量。
- 如果机械臂出现危险动作，立即 Ctrl+C，必要时断电，然后检查校准和模型是否对应。